

IA 642 Defensive Security

Week 05 Bonus / Extra Credit Lab

Seeing the Graph: Visualizing and Sharing Meridian's STIX Intelligence

Cyber Threat Intelligence Tooling — STIX 2.1 · stix2viz · TAXII 2.1

Repository: github.com/unaishshazan/ia642-week05-lab51-cti/Lab5_Bonus_Assignment

Author	Unais Ali	EID	E02805019
Course	IA 642 Defensive Security	Week	05, Bonus Lab
Tools	stix2-validator, stix2viz, medallion, taxii2-client	Date	October 2026

Lab overview

This bonus lab takes the closed Week 04 Kerberoasting / `svcupdate.bin` case and runs the full CTI tooling loop: author a STIX 2.1 bundle, validate it against the OASIS schema (not just `json.tool`), render it as a node-and-edge graph in OASIS `stix2viz`, serve it from a live TAXII 2.1 `medallion` collection, pull it back with `taxii2-client`, then extend the feed and re-verify end to end.

Host	Role
Analysis host (Python 3.11)	Authoring, validation, stix2viz HTTP server, medallion TAXII server, taxii2-client consumer

1. Part A — Assemble and validate the bundle

Step 1 — Bundle contents

The hand-authored bundle `meridian_week4_intel.json` contains six STIX 2.1 objects:

1. Identity SDO — Meridian Health Network SOC
2. Malware SDO — MeridianLoader (`is_family=true`, type `downloader`)
3. Attack-Pattern SDO — Kerberoasting with ATT&CK T1558.003
4. Indicator SDO — pattern [`file:name = 'svcupdate.bin'`]
5. Relationship SRO — `indicates` (indicator → malware)
6. Relationship SRO — `uses` (malware → attack-pattern)

`python -m json.tool` reported well-formed JSON before schema validation.

Step 2 — OASIS stix2-validator

Validator run used STIX 2.1 schemas from the OASIS `cti-stix2-json-schemas` repository (`stix2.1` branch). The pip wheel alone does not ship schemas; pointing `stix2_validator --version 2.1 -s <schemas>` is required for a real schema check.

Evidence #1. Full `stix2_validator` output on the original six-object bundle:

```
=====
[-] Results for: meridian_week4_intel.json
[+] STIX JSON: Valid
    [!] Warning: attack-pattern--3f1a2b4c-0003-4a7b-8c9d-000000000003:
        {302} External reference 'mitre-attack' has a URL but no hash.
```

Interpretation. The bundle is schema-valid with zero errors. The single residual SHOULD-warning {302} is the expected mitre-attack external-reference hash advisory called out in the lab

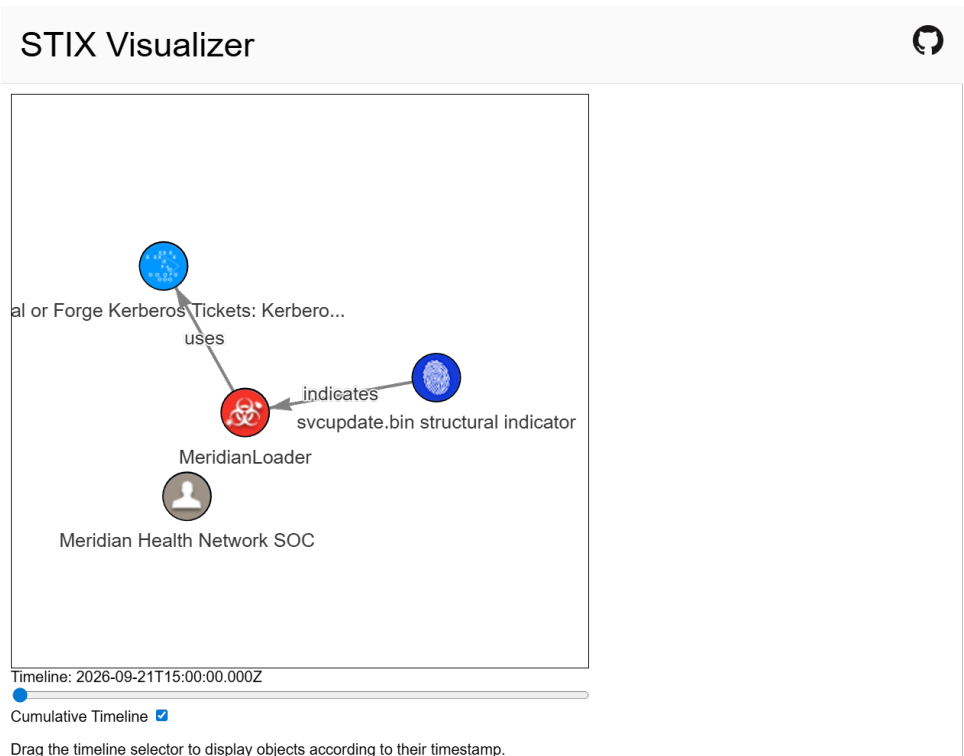
prompt; MITRE's own ATT&CK references typically omit a hash as well, so it was left as-is. No other warnings remained.

2. Part B — Visualize the bundle as a graph

Step 3 — OASIS stix2viz

cti-stix-visualization was served locally with `python -m http.server 8080` and the original bundle was loaded in the browser at `http://127.0.0.1:8080/`.

Evidence #2. Rendered graph (OASIS stix2viz) — original six-object bundle:



Graph reading.

- **Nodes (SDOs):** 4 — Identity, Malware (**MeridianLoader**), Attack-Pattern (Kerberoasting), Indicator (**svcupdate.bin**).
- **Edges (SROs):** 2 — **indicates** and **uses**.
- **Most-connected node:** Malware **MeridianLoader** (degree 2). The Identity node is present but unlinked (no `created_by_ref` / authorship edges in this minimal bundle).

Malware node property panel contents:

```
type: malware
spec_version: 2.1
id: malware--3f1a2b4c-0002-4a7b-8c9d-000000000002
created / modified: 2026-09-21T15:00:00.000Z
name: MeridianLoader
is_family: true
malware_types: ["downloader"]
description: Loader recovered as svcupdate.bin from BILL-WS-14.
  Detected by YARA rule Meridian_SvcUpdate_Loader.
```

Critical thinking #1

In this six-object bundle the **Malware** SDO (**MeridianLoader**) is the hub: both SROs touch it (indicates inbound from the Indicator; uses outbound to the Attack-Pattern). Relationship direction matters because STIX edges are typed and oriented. Reading `source_ref` → `target_ref` correctly distinguishes “the indicator indicates the malware” from the reverse claim, and “the malware uses Kerberoasting” from implying that the technique somehow contains the malware. Wrong direction would invert the analytic meaning even if the same two nodes were drawn next to each other.

3. Part C — Stand up a TAXII 2.1 server

Steps 4–6 — medallion

medallion v3.0.0 was installed and configured with a memory backend whose `server_data.json` wraps the six (later eight) STIX objects in one Collection. Discovery was confirmed with Basic auth and the mandatory TAXII 2.1 Accept header:

```
curl -s -u analyst:MeridianLab2026! \
  -H "Accept: application/taxii+json;version=2.1" \
  http://127.0.0.1:5000/taxii2/
```

Evidence #3. Discovery-endpoint JSON response:

```
{
  "api_roots": [
    "http://127.0.0.1:5000/meridian/"
  ],
  "contact": "soc@meridian-lab.test",
  "default": "http://127.0.0.1:5000/meridian/",
  "description": "Week 05 bonus lab Week 04 Kerberoasting incident intel",
  "title": "Meridian SOC TAXII Server"
}
```

Config notes: both `module` and `module_class` are required under `backend` for medallion’s `connect_to_backend()`; the README example that omits `module` fails. Host/port used 127.0.0.1:5000 (not localhost) to avoid IPv6 bind mismatches.

4. Part D — Pull with a real TAXII client

Step 7 — taxii2-client

```
from taxii2client.v21 import Server
server = Server("http://127.0.0.1:5000/taxii2/",
               user="analyst", password="MeridianLab2026!")
api_root = server.api_roots[0]
collection = api_root.collections[0]
bundle = collection.get_objects()
```

Evidence #4. Client script output — six objects retrieved, IDs match Step 1:

```
API root title: Meridian Intel Sharing
Collection: Meridian Week 04 Incident Intel - 6b9a7e2c-1234-4a11-9b11-abcdefabcdef
Objects retrieved: 6
- identity identity--3f1a2b4c-0001-4a7b-8c9d-000000000001
- malware malware--3f1a2b4c-0002-4a7b-8c9d-000000000002
- attack-pattern attack-pattern--3f1a2b4c-0003-4a7b-8c9d-000000000003
- indicator indicator--3f1a2b4c-0004-4a7b-8c9d-000000000004
- relationship relationship--3f1a2b4c-0005-4a7b-8c9d-000000000005
- relationship relationship--3f1a2b4c-0006-4a7b-8c9d-000000000006
```

Critical thinking #2

Step 7 never re-opened `meridian_week4_intel.json`; the client spoke only TAXII to medallion. That proves the server's Collection actually serialized and served the published objects over the wire under TAXII 2.1 auth and media types—not merely that a local file is self-consistent. Re-reading your own file only proves you can parse what you already wrote. A partner's tooling would never see that file; it would see whatever the server returns. Matching IDs/types on the client therefore validates the publish path (config, backend load, discovery, collection `get_objects`), which is the operational claim a shared feed must make.

5. Part E — Challenge: extend the feed

Step 8 — New observation

New fact chosen: the content hash of the recovered loader sample (SHA-256 `3f77970a8c10f1af034e984b3b06f76db1708d377e8c3f9085af95823c9e2d73` from the Week 05 static triage of `svcupdate.bin`). Added objects:

- Indicator SDO `indicator--3f1a2b4c-0007-...` with pattern `[file:hashes.'SHA-256' = '3f77970a...']`
- Relationship SRO `relationship--3f1a2b4c-0008-...` with `relationship_type = "indicates"` linking that indicator to `MeridianLoader`

The updated bundle was re-validated, `server_data.json` regenerated, medallion restarted, the client re-pulled, and `stix2viz` reloaded.

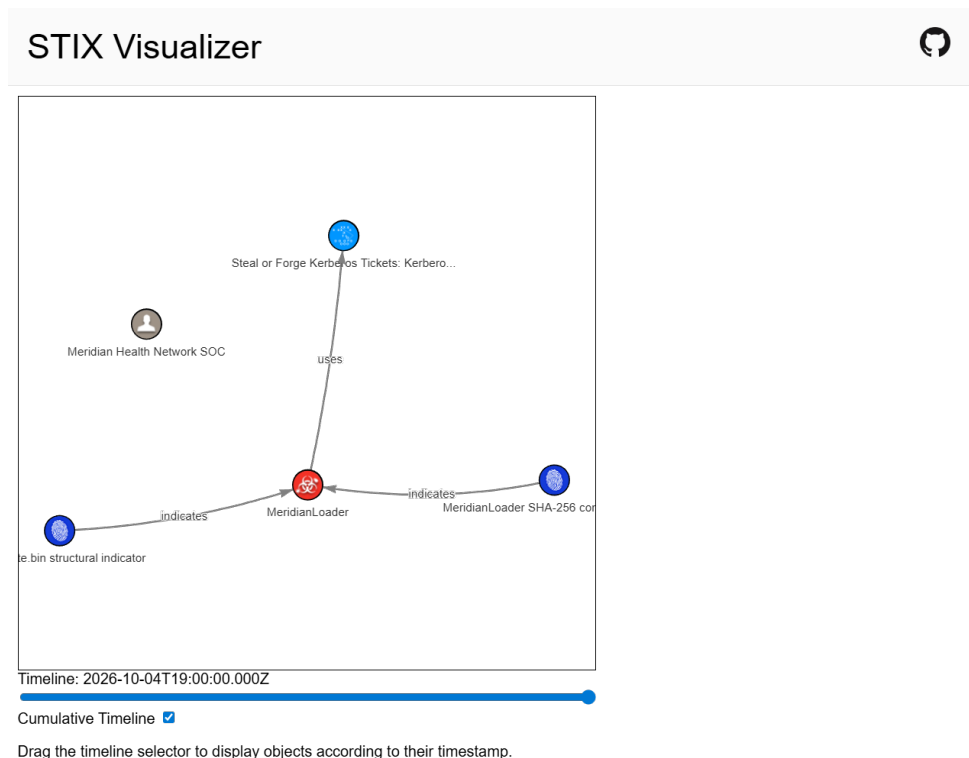
Evidence #5. Passing validator on the updated (8-object) bundle:

```
=====
[-] Results for: meridian_week4_intel.json (updated)
[+] STIX JSON: Valid
    [!] Warning: attack-pattern--3f1a2b4c-0003-4a7b-8c9d-000000000003:
        {302} External reference 'mitre-attack' has a URL but no hash.
```

Evidence #6. Updated client pull — eight objects, including the new indicator and relationship:

```
API root title: Meridian Intel Sharing
Collection: Meridian Week 04 Incident Intel - 6b9a7e2c-1234-4a11-9b11-abcdefabcdef
Objects retrieved: 8
- identity identity--3f1a2b4c-0001-4a7b-8c9d-000000000001
- malware malware--3f1a2b4c-0002-4a7b-8c9d-000000000002
- attack-pattern attack-pattern--3f1a2b4c-0003-4a7b-8c9d-000000000003
- indicator indicator--3f1a2b4c-0004-4a7b-8c9d-000000000004
- relationship relationship--3f1a2b4c-0005-4a7b-8c9d-000000000005
- relationship relationship--3f1a2b4c-0006-4a7b-8c9d-000000000006
- indicator indicator--3f1a2b4c-0007-4a7b-8c9d-000000000007
- relationship relationship--3f1a2b4c-0008-4a7b-8c9d-000000000008
```

Evidence #7. `stix2viz` after extension — one additional node and one additional edge:



The graph moved from **4 nodes / 2 edges** to **5 nodes / 3 edges**. The new fingerprint-style Indicator node (**MeridianLoader** SHA-256...) connects into MeridianLoader with an **indicates** edge; MeridianLoader's hub degree rose from 2 to 3. The timeline includes the new objects' timestamp 2026-10-04T19:00:00.000Z.

Critical thinking #3 (challenge graded)

I chose an **Indicator** SDO because the new fact is a detection pattern: the SHA-256 of the recovered **svcupdate.bin** loader. That is not a new malware family, not an ATT&CK technique, and not a raw **file** SCO. Indicator is the STIX type that carries a machine-checkable **pattern** detection tools can load. A lone file SCO would only say the hash was seen; a second Malware object would falsely imply another family; an Attack-Pattern would mislabel a hash as a technique.

I linked it with **relationship_type** = "indicates" (Indicator → MeridianLoader), not **related-to** or **uses**. **indicates** is the STIX meaning that a pattern match is evidence of that malware. **uses** would claim the hash employs Kerberoasting; **related-to** would be a vague link with no detection force. From that SRO a partner may conclude a SHA-256 hit attributes to MeridianLoader—which the Indicator alone does not authorize, because without the relationship the hash is an orphaned sighting with no named malware target. The directed SRO is what makes the hash shared, actionable intel.

6. End-to-end workflow

Author bundle → Validate → stix2viz → medallion → taxii2-client

The client retrieval matching the authored IDs closes the loop: what was published is what a partner would receive.